

CRO × CLAUDE CODE • 50 MIN

Como usar o *Claude Code* para análise de dados

Lucian Fialho — Métricas Boss

Quantas horas da semana você passou
preparando dado antes de conseguir
analisar de verdade?

0 SEU STACK DE CRO HOJE



Google Analytics 4

Sessions · events
conversions · funnels



Google Ads

Clicks · impressions
spend · ROAS



Meta Ads

Reach · CPM
attributed conversions



Search Console

Queries · impressions
CTR · position



VTEX / Shopify

Orders · revenue
SKUs · sessions



Hotjar

Heatmaps · recordings
surveys



Google Tag Manager

Tags · triggers
variables



VWO

Test results · variants
goals · segments



AB Tasty

Experiments · winners
uplift · confidence



Optimizely

Variations · metrics
statistical significance

+ Convert · BQ
· ERP · planilha

Nenhum deles conversa entre si.

Como as empresas resolvem isso hoje

A SOLUÇÃO CLÁSSICA



BigQuery



Databricks



Snowflake



dbt

Centraliza tudo. Dado limpo. Queries em SQL.
Funciona.

0 CUSTO REAL

- 1 Engenheiro de dados (ou time inteiro) pra montar e manter o pipeline
- 3 a 6 meses de setup antes do primeiro dado limpo disponível
- ∞ Você ainda precisa de alguém pra escrever as queries quando quiser uma análise nova

E quem não tem time de dados?

Não estamos acabando com o warehouse. Estamos acabando com a *fila de tickets*.

ANTES

- analista → abre ticket com a pergunta
- eng. dados → escreve a query em SQL
- 3-5 dias → analista recebe a resposta
- analista → percebe que precisava de outro filtro e abre novo ticket

DEPOIS

- analista → pergunta em linguagem natural
- Claude Code → gera e executa SQL no seu BQ / DuckDB
- 20 segundos → resposta, interpretação, próximo passo

SQL é a linguagem mais representada nos dados de treino dos LLMs.

Claude Code sabe BigQuery, Snowflake, dbt nativamente. O warehouse continua lá — a fila de tickets vai embora.

ChatGPT, Claude Desktop ou *Claude Code*?

ChatGPT

- Cola texto ou sobe arquivo
- Não acessa sua máquina nem suas APIs
- 128k de contexto — você samplea o dado
- Cada sessão começa do zero

Chat de texto

Claude Desktop + MCP

- Pode fazer análise — com MCP configurado manualmente por tool
- MCP injeta schema de TODOS os tools no contexto da sessão
- ~20x mais tokens por análise vs CLI — custo por acidente de arquitetura
- Sem skills, hooks ou loop agêntico nativo

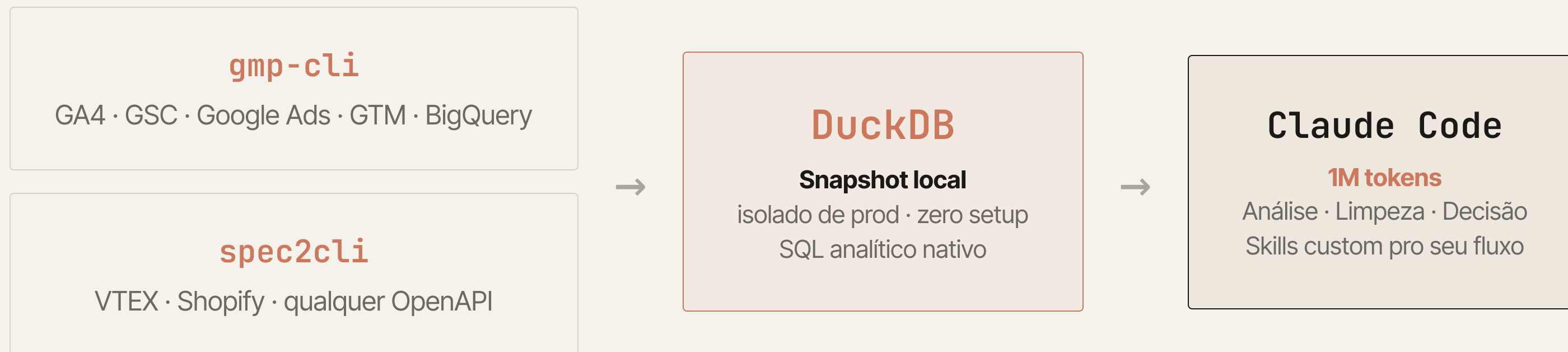
Funciona, mas caro

Claude Code

- Roda no terminal, dentro da pasta do projeto
- Lê arquivos, roda scripts, chama APIs
- 1M de contexto — dataset completo, sem amostragem
- Skills e hooks custom pro seu fluxo

Agente de análise

O stack que eu *recomendo*



Open source · github.com/lucianfialho · funciona com qualquer warehouse existente

@LUCIANFIALHO

gmp-cli Google Marketing Platform no terminal

CONECTA COM

**Google Analytics 4**
eventos • sessões • conversões • funis

**Search Console**
queries • impressões • CTR • posição

**Google Ads**
clicks • spend • ROAS • keywords

**BigQuery**
GA4 raw • warehouse existente

EXEMPLO

```
gmp ga report \  
  --days 30 \  
  --metrics sessions,conversions \  
  --dims device,channel  
  
gmp gsc queries \  
  --site metricasboss.com.br \  
  --days 28
```

Output vai direto pro DuckDB. Claude Code cruza GA4 + GSC + Ads numa query.

SKUNKWORKS • NEWSLETTER

MCP vs CLI Token Benchmark: Google Analytics

Um benchmark replicável para comparação de custos de tokens entre o Google Analytics MCP vs **gmp-cli**

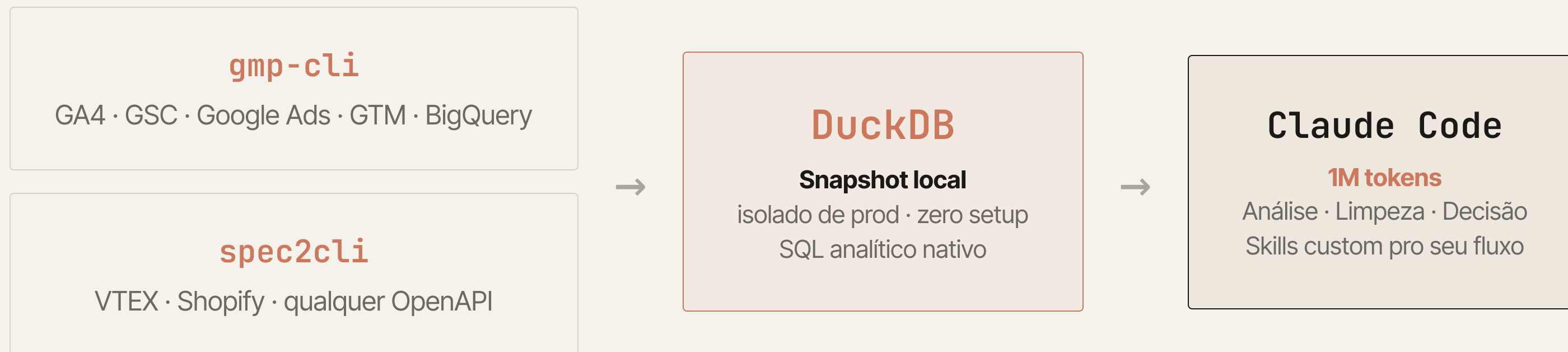
Lucian Fialho • Abr 06, 2026

O benchmark que gerou *o número.*

Comparação replicável entre Google Analytics MCP e gmp-cli. Metodologia aberta, dados reais, resultado: CLI consome até 20x menos tokens pras mesmas análises.

lucianfialho.substack.com →

O stack que eu *recomendo*



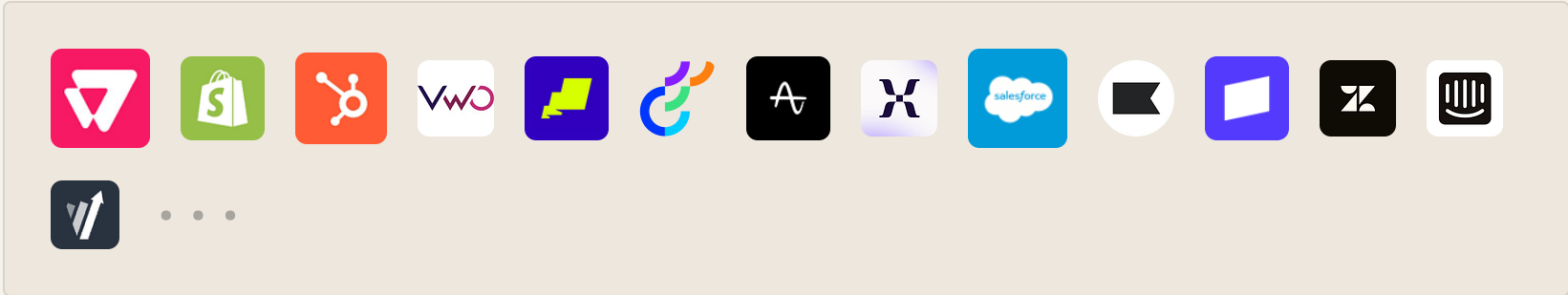
Open source · github.com/lucianfialho · funciona com qualquer warehouse existente

@LUCIANFIALHO

spec2cli

Qualquer API vira CLI em segundos

SE TEM OPENAPI URL → VIRA CLI



O FLUXO

```
# só precisa da URL do spec
spec2cli \
  --spec https://api.vtex.com/openapi \
  orders list \
  --status invoiced \
  --date-from 2026-04-01
```

*"Se tem OpenAPI URL, é só apontar.
Sem instalar SDK. Sem escrever código."*

SKUNKWORKS • NEWSLETTER

Como consumir qualquer API no Claude Code sem estourar o contexto

Como o CLI discovery reduz em **18x o custo de tokens** na hora de apontar o Claude pra uma API externa

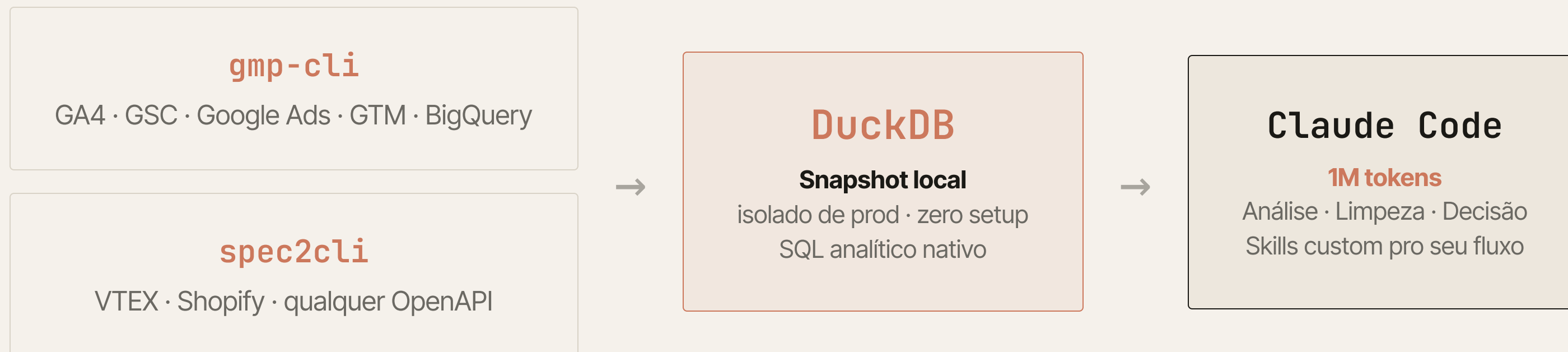
Lucian Fialho • Abr 27, 2026

Escrevi o experimento completo.
Está na newsletter.

MCPs consomem 1.3–21x mais tokens que CLI pras mesmas ações. A causa é arquitetural — o schema completo vai pro contexto antes de qualquer ação útil.

lucianfialho.substack.com →

O stack que eu *recomendo*



Open source · github.com/lucianfialho · funciona com qualquer warehouse existente

@LUCIANFIALHO

DuckDB não é só cache. É uma *camada de proteção*.



Se você tem um Data Lake — ingere o snapshot direto no DuckDB. Custo de 1 query de ingestão. Análises ilimitadas sem tocar no lake de novo.

ALTERNATIVA SEM INSTALAR DUCKDB
Python + **pandas** — pré-instalado no macOS.
Mesmo resultado, zero dependência extra.

Claude *Code*

Agente de análise no topo do stack.
Lê os dados do DuckDB, interpreta, age.

- 01 Roda no terminal, dentro da pasta do projeto. Acessa arquivos, executa scripts.
- 02 Skills custom — você escreve em markdown, ele aprende o seu fluxo específico.
- 03 Hooks de validação — guardrails como código, não como instrução.
- 04 /schedule e /loop — transforma análise pontual em agente recorrente.

Como eu *inicializo* na prática

sessão de análise – modo completo

```
claude \  
  --plugin-dir ~/Code/claude-dev-pipeline \  
  --dangerously-skip-permissions \  
  --remote-control \  
  --append-system-prompt "no suggarcoat"
```

plugin-dir → skills e hooks do pipeline skip-permissions → sem confirmações remote-control → monitoro do celular append-system-prompt → instrução fixa de tom

pipeline automatizado – não-interativo

```
cat events.csv | claude -p \  
  --append-system-prompt "analise os dados de CR0, responda em PT-BR" \  
  --max-turns 5 \  
  --output-format json
```

-p → print mode, sai após resposta max-turns → evita loop infinito output-format json → parse fácil no pipeline

Como você *treina* o Claude pro seu fluxo

Skills

Arquivos
markdown

Você escreve as instruções uma vez. O Claude executa sempre que você chama o skill.

Slash Commands

Atalhos
de fluxo

Skills invocadas com */nome* dentro do Claude.
Um comando dispara uma sequência inteira de ações.

Hooks

Scripts
automáticos

Shell scripts que rodam em momentos específicos — antes de uma ação, depois de um commit, no início da sessão.

Tudo é arquivo. Você commita no Git, compartilha com o time, versiona como código.

Skills

instruções que o Claude aprende uma vez e executa sempre

ESTRUTURA DE UM SKILL

```
srm-check.md

---

name: srm-check
description: Valida SRM antes
             de declarar resultado
---

## Tarefa
Buscar no DuckDB a distribuição
de usuários por variante...
```

É markdown puro. Frontmatter define o nome e a descrição. O corpo é a instrução que o Claude vai seguir.

SKILLS QUE VAMOS USAR HOJE

- `/srm-check`

Valida distribuição de tráfego antes de qualquer análise
- `/post-test-segments`

Segmentação pós-teste por device, canal, novo/recorrente
- `/cro-weekly-pipeline`

Pipeline completo: puxa → limpa → SRM → segmenta → reporta
- `/heuristic-scan`

11-point checklist visual via agent-browser

Onde encontrar mais *skills*



Jesse Storimer
@obra

superpowers

Skills para brainstorming, code review, TDD, debugging e execução de planos. O ecossistema mais completo de skills disponível.



github.com/obra/superpowers



Matt Pocock
@mattpocock

skills

Skills focadas em TypeScript e desenvolvimento frontend. Boa referência de como estruturar skills técnicas de qualidade.



github.com/mattpocock/skills



Anthropic
oficial

claude-plugins-official

Diretório oficial de plugins e skills mantido pela Anthropic. Instale via **claude plugin marketplace**.



github.com/anthropics/claude-plugins-official

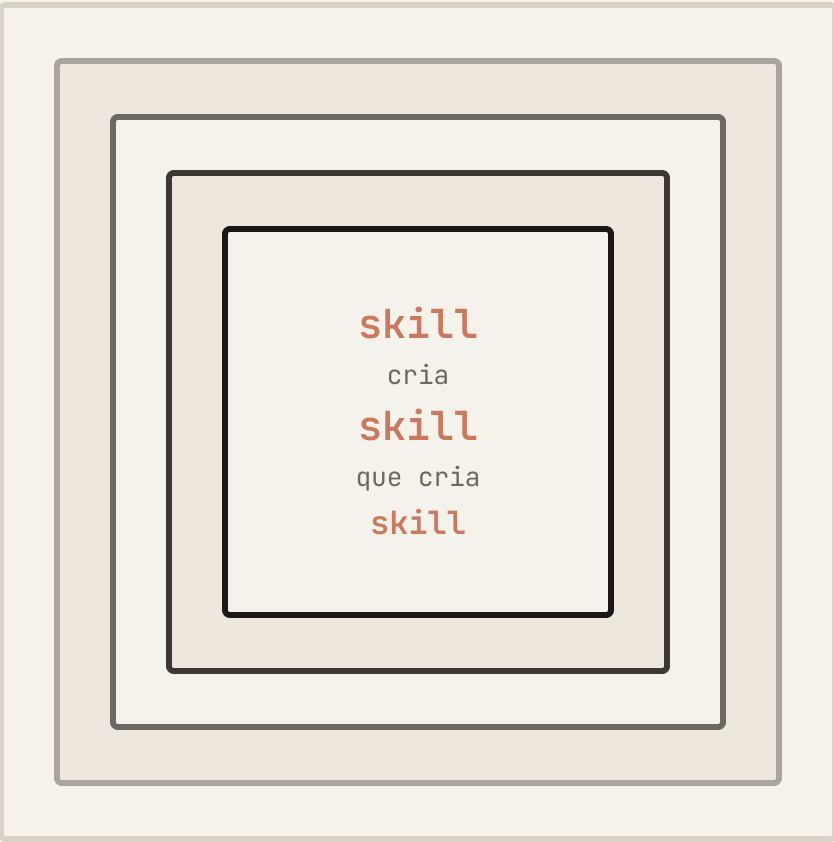
Anthropic · skill-creator

A skill que cria *skills.*

Você descreve o que quer que o Claude faça. Ela escreve o SKILL.md por você — com frontmatter, instruções e exemplos corretos.



`github.com/anthropics/skills`
`/skill-creator`



"We need to go deeper."

/comando

um atalho que dispara uma sequência inteira

SEM SLASH COMMAND

"Me ajude a verificar se o teste A/B tem SRM, calcule o chi-quadrado, verifique a distribuição de tráfego entre as variantes usando a tabela events_clean no DuckDB, e me diga se posso declarar vencedor..."

Você reescreve toda vez. Sem garantia de consistência.

COM SLASH COMMAND

/srm-check

Uma palavra. Sempre a mesma lógica. Sempre o mesmo output estruturado.

Slash commands são a interface entre você e o Claude. Quanto mais específicos pro seu negócio, mais valor você extrai.

Hooks

guardrails que rodam antes do Claude agir

MOMENTOS DISPONÍVEIS

- SessionStart

Início ou retomada de sessão
- UserPromptSubmit

Antes do Claude processar sua mensagem
- PreToolUse ← usamos hoje

Antes de executar qualquer ferramenta (Bash, Edit, Read)
- PostToolUse

Após executar uma ferramenta com sucesso
- Stop

Quando Claude termina de responder
- PreCompact

Antes de compactar o contexto da sessão

28 hooks no total – docs.claude.ai/hooks

O HOOK QUE VAMOS USAR HOJE

```
settings.json – PreToolUse

{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "command": "bash validate.sh"
      }]
    }]
  }
}
```

Antes de qualquer Bash rodar, validate.sh verifica se SRM check foi feito. Se não foi, bloqueia. **Não é prompt. É código.**

Onde encontrar mais *hooks*



Lucian Fialho
@lucianfialho

awesome-claude-code-hooks

Lista curada de hooks para Claude Code — PreToolUse, SessionStart, UserPromptSubmit e mais. Contribuições abertas.



github.com/lucianfialho/awesome-claude-code-hooks

Hooks são *código*.

Shell scripts. Qualquer linguagem. Qualquer lógica. Se você sabe escrever um script bash, você sabe escrever um hook.

PreToolUse → valida antes

UserPromptSubmit → adiciona contexto

Stop → notifica quando termina

SessionStart → inicializa ambiente

PARA QUEM NÃO É DEV

Bonus

agent-browser — Claude Code abre o
browser, analisa páginas e gera hipóteses de
teste

@LUCIANFIALHO

agent-*browser*

CLI em Rust para automação de browser feito pra agentes de IA. Claude Code abre uma página, navega, interage e analisa — tudo autonomamente.

```
npx skills add vercel-labs/agent-browser
```

Navegação

Abre URLs, clica, preenche forms, navega entre páginas

Screenshots + pixel-diff

Screenshot anotado, comparação visual entre variantes

Network monitoring

Captura requests de rede — valida se GTM disparou

Accessibility tree

Mapeia todos os elementos interativos com referências (@e1, @e2...)

De "o que medir" a *plano pronto*

1. ABRE E ANALISA

`agent-browser open URL`

Captura screenshot, mapeia elementos, identifica CTAs, forms, navegação

2. CRUZA COM GTM

`gmp gtm list-tags`

Lista tags ativas. Claude compara com elementos da página — identifica o que não está sendo rastreado

3. GERA O PLANO

`/measurement-plan`

Output: eventos a criar, dataLayer keys, triggers GTM e KPIs por objetivo de negócio

Da página à *hipótese* em segundos

```
agent-browser open https://loja.com/checkout
agent-browser screenshot --annotate
agent-browser snapshot -i
/generate-ab-hypothesis
```

Claude analisa o screenshot anotado + accessibility tree e gera hipóteses com: elemento específico, comportamento esperado, métrica de sucesso e tamanho de amostra estimado.

OUTPUT GERADO

H1 – CTA acima da dobra

Mover botão "Comprar" para thumb zone em mobile deve aumentar CR em +15-25% em mobile. Métrica: purchase / session por device.

H2 – Redução de campos

Form tem 7 campos. Remover 3 opcionais deve reduzir abandono em +20%. Métrica: form_submit / form_start.

H3 – Contraste do botão

Ratio atual: 3.2:1 (falha WCAG AA). Aumentar para 4.5:1 deve melhorar clicks em +8%. Impacto maior em usuários 55+.

GTM disparou? *Confirma agora.*

O FLUXO DE VALIDAÇÃO

```
terminal

# abre página e monitora requests
agent-browser open https://loja.com
agent-browser network start

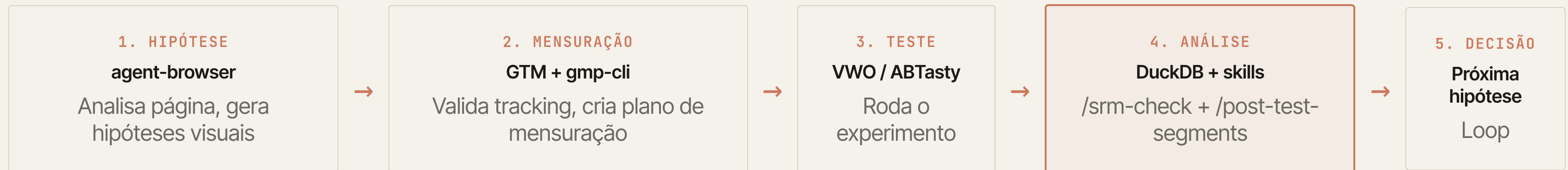
# clica no botão de compra
agent-browser click @e12

# verifica se GTM disparou
agent-browser network stop
agent-browser list-network-requests
```

O QUE CLAUDE VERIFICA

- ✓ Request pra Google Analytics com evento purchase disparado
- ✓ DataLayer contém transaction_id e revenue
- ✗ Tag de Meta Ads não disparou — trigger está configurado só pra desktop
- ✗ add_to_cart ausente no DL — produto X não tem variante configurada

O ciclo *completo* de CRO



Cada etapa tem uma skill. O ciclo inteiro pode ser agendado com [/schedule](#).

DE ANÁLISE MANUAL PARA AGENTE RECORRENTE

```
/schedule every monday at 8am:  
/cro-weekly-pipeline
```

Pipeline
semanal

Toda segunda às 8h: puxa dados frescos, limpa bots, verifica SRM em todos os testes ativos, gera relatório segmentado. Você chega com a análise pronta.

```
/loop 2h: /srm-check
```

Plantonista
24/7

De 2 em 2 horas: verifica integridade dos testes rodando. Se detectar SRM ou anomalia, avisa antes de você declarar um vencedor errado.

4 SKILLS • MARKDOWN • GIT CLONE • FUNCIONA AGORA

/srm-check

SRM Check

Valida distribuição de tráfego antes de qualquer análise. χ^2 automático, veredicto claro.

/post-test-segments

Segmentação pós-teste

Quebra o resultado por device, novo/recorrente e canal. Revela onde o lift realmente aconteceu.

/cro-weekly-pipeline

Pipeline semanal

Puxa → limpa → SRM → segmenta → reporta. Um comando, relatório completo toda segunda.

/heuristic-scan

Heuristic Scan

agent-browser roda 11 checks (desktop + mobile). Hipóteses visuais correlacionadas com o lift quantitativo.

Ficou interessado? *Me segue.*

NEWSLETTER — SKUNKWORKS

Experimentos com Claude Code — toda semana

Benchmarks, casos reais, ferramentas novas. O que eu testo antes de palestrar.



lucianfialho.substack.com

@LUCIANFIALHO

LINKEDIN



Lucian Fialho
Métricas Boss

Analytics, CRO e experimentos com Claude Code. Conexões relevantes bem-vindas.



linkedin.com/in/lucianfialho

Obrigado.

@lucianfialho

metricasboss.com.br